

ISTIC — License 3 MIAGE

Systèmes

20 décembre 2018, 8 :00–10 :00

Examen, 1e session

Seules les notes de cours, TD et TP sont autorisées.

Les réponses aux questions doivent être courtes et précises.

Le barème est indicatif.

Q1. (2pt) Quelle est la différence entre un multiplexeur et un décodeur ? Dans quelles circonstances utilise-t-on chacun d'entre eux ?

Q2. (1pt) Les processeurs modernes utilisent des pipelines avec 7 à 10 étages. Expliquez la raison pour laquelle rajouter davantage d'étages n'est pas forcément une bonne idée.

Q3. (3pt) Que réalise le micro-programme suivant ? Quelle instruction assembleur implémente-t-il ? Expliquez votre réponse.

```
1 xxx1: MAR = SP
2 xxx2: H = TOS
3 xxx3: MDR = TOS = TOS + H; wr; goto main1
```

Q4. (3pt) Traduisez le morceau de programme Java ci-dessous en assembleur IJVM (celui de la machine Mic1MVV vue en cours). Vous pouvez ignorer la déclaration de la variable `i` et vous concentrer sur la partie de code. On suppose que la fonction `afficher()` a été déclarée préalablement.

```
1 int i;
2
3 (...)
4
5 for (i=0; i<3; i++) {
6     afficher(i);
7 }
```

Q5. (1pt) Expliquez la différence entre la localité spatiale et la localité temporelle.

Q6. (1pt) Donnez un exemple concret de programme avec beaucoup de localité spatiale mais très peu de localité temporelle.

Q7. (1pt) Si l'on décidait de concevoir une machine dédiée uniquement à l'exécution du programme de la question précédente, quelle serait une bonne valeur pour la taille des lignes de cache mémoire ? Expliquez votre réponse.

Q8. (1pt) Que fait l'appel système `kill()` ? A quoi sert-il ?

- Q9. (2pt) Le programme helloworld a décidé de créer une variable `int toto` à l'adresse mémoire 1000. Par ailleurs, le programme `projetetudiant` a également décidé de créer une variable `String toto` à l'adresse mémoire 1000. Expliquez le mécanisme qui permet aux deux programmes de s'exécuter simultanément sans se gêner mutuellement.
- Q10. (2pt) Le programme dont on donne le pseudo-code ci-dessous comprend une "race condition" si plusieurs processus l'exécutent simultanément. Montrez un exemple d'exécution incorrecte qui illustre cette race condition.

```

1  const int TAILLE_CHAINE = 1024;
2  String chaine;      /* la variable Chaine est en memoire partagee */
3
4  int main() {
5      int longueur;
6      char lettre;
7
8      lettre = lire_une_lettre_du_clavier();
9      longueur = taille_de_la_chaine(chaine);
10     ecrire_la_variable_lettre_a_l'emplacement_longueur+1_dans_la_chaine
11     longueur = longueur + 1;
12
13     return 0;
14 }

```

- Q11. (2pt) Pour tenter de résoudre cette race condition, un programmeur propose le programme suivant. La variable partagée `verrou` vaut 0 avant le début des exécutions. Est-ce que la "race condition" est toujours possible? Expliquez votre réponse.

```

1  const int TAILLE_CHAINE = 1024;
2  int verrou;      /* la variable verrou est en memoire partagee */
3  String chaine;   /* la variable chaine est en memoire partagee */
4
5  int main() {
6      int longueur;
7      char lettre;
8
9      while (verrou != 0) { } /* attendre que le verrou se libere */
10     verrou = 1;             /* verrouiller le verrou */
11     lettre = lire_une_lettre_du_clavier();
12     longueur = taille_de_la_chaine(chaine);
13     ecrire_la_variable_lettre_a_l'emplacement_longueur+1_dans_la_chaine
14     longueur = longueur + 1;
15     verrou = 0;             /* deverrouiller le verrou */
16
17     return 0;
18 }

```

- Q12. (1pt) Nous considérons un système de fichier de type FAT. Dans une nouvelle version de ce système de fichier on voudrait ajouter une méta-donnée supplémentaire au sujet de chaque fichier pour indiquer le nombre de fois où ce fichier a été ouvert depuis sa création. Expliquez à quel endroit dans le système de fichier nous devons stocker cette information supplémentaire.

— fin —